1. **Задание**

   Выполнить на выбор один из вариантов:

   - Шейдер с сохранением данных в текстуре,

   - Процедурный, текстурный шейдер,

   - Традиционные шейдер,

   - Фрагментный шейдер,

   - Вершинный шейдер.

   Будет выполнен вершинный шейдер

2. **Листинг программы**

```
#include <GL/glew.h>
#include <GL/freeglut.h>
#include <iostream>
#include <string>
#include <fstream>
#include <sstream>

using namespace std;

// Функция для загрузки шейдера
GLuint LoadShader(const char* fragPath) {
    string fragmentCode;
    ifstream fragShaderFile;

    fragShaderFile.open(fragPath);
    stringstream fragShaderStream;
    fragShaderStream << fragShaderFile.rdbuf();

    fragShaderFile.close();
    fragmentCode = fragShaderStream.str();

    const char* fragShaderCode = fragmentCode.c_str();

    // Компиляция
    GLuint fragment;

    // Шейдер
    fragment = glCreateShader(GL_FRAGMENT_SHADER);
    glShaderSource(fragment, 1, &fragShaderCode, NULL);
    glCompileShader(fragment);

    // Шейдер программа
    GLuint shaderProgram = glCreateProgram();
    glAttachShader(shaderProgram, fragment);
    glLinkProgram(shaderProgram);

    // Удаляем шейдеры, так как они теперь встроены в программу
    glDeleteShader(fragment);

    return shaderProgram;
}

// Массив вершин для отображения треугольника
GLfloat vertices[] = {
    -0.5f, -0.5f, 0.0f,  // Вершина 1
     0.5f, -0.5f, 0.0f,  // Вершина 2
     0.0f,  0.5f, 0.0f   // Вершина 3
```

```
};

// Используем наборы вершин
GLuint VBO, VAO;
GLuint shaderProgram;

void init() {
    // Компилируем шейдеры
    shaderProgram = LoadShader("fragment_shader.glsl");

    // Создаем VAO и VBO
    glGenVertexArrays(1, &VAO);
    glGenBuffers(1, &VBO);

    // Привязываем VAO
    glBindVertexArray(VAO);

    // Привязываем VBO и загружаем данные вершин
    glBindBuffer(GL_ARRAY_BUFFER, VBO);
    glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices, GL_STATIC_DRAW);

    // Указываем OpenGL, как интерпретировать данные вершин
    glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 3 * sizeof(float), (void*)0);
    glEnableVertexAttribArray(0);

    // Отвязываем VBO и VAO
    glBindBuffer(GL_ARRAY_BUFFER, 0);
    glBindVertexArray(0);
}

void display() {
    // Очищаем экран
    glClear(GL_COLOR_BUFFER_BIT);

    // Используем шейдерную программу
    glUseProgram(shaderProgram);

    // Привязываем VAO
    glBindVertexArray(VAO);

    // Отрисовываем треугольник
    glDrawArrays(GL_TRIANGLES, 0, 3);

    // Отвязываем VAO
    glBindVertexArray(0);

    glFlush();
}

int main(int argc, char** argv) {
    // Инициализация GLUT
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(800, 600);
    glutCreateWindow("LR8");

    glewInit();

    // Инициализация шейдера
    init();

    // Устанавливаем функцию для рендеринга
    glutDisplayFunc(display);

    // Входим в основной цикл
```

```
  glutMainLoop();

  return 0;
}
```

3. Пример работы программы

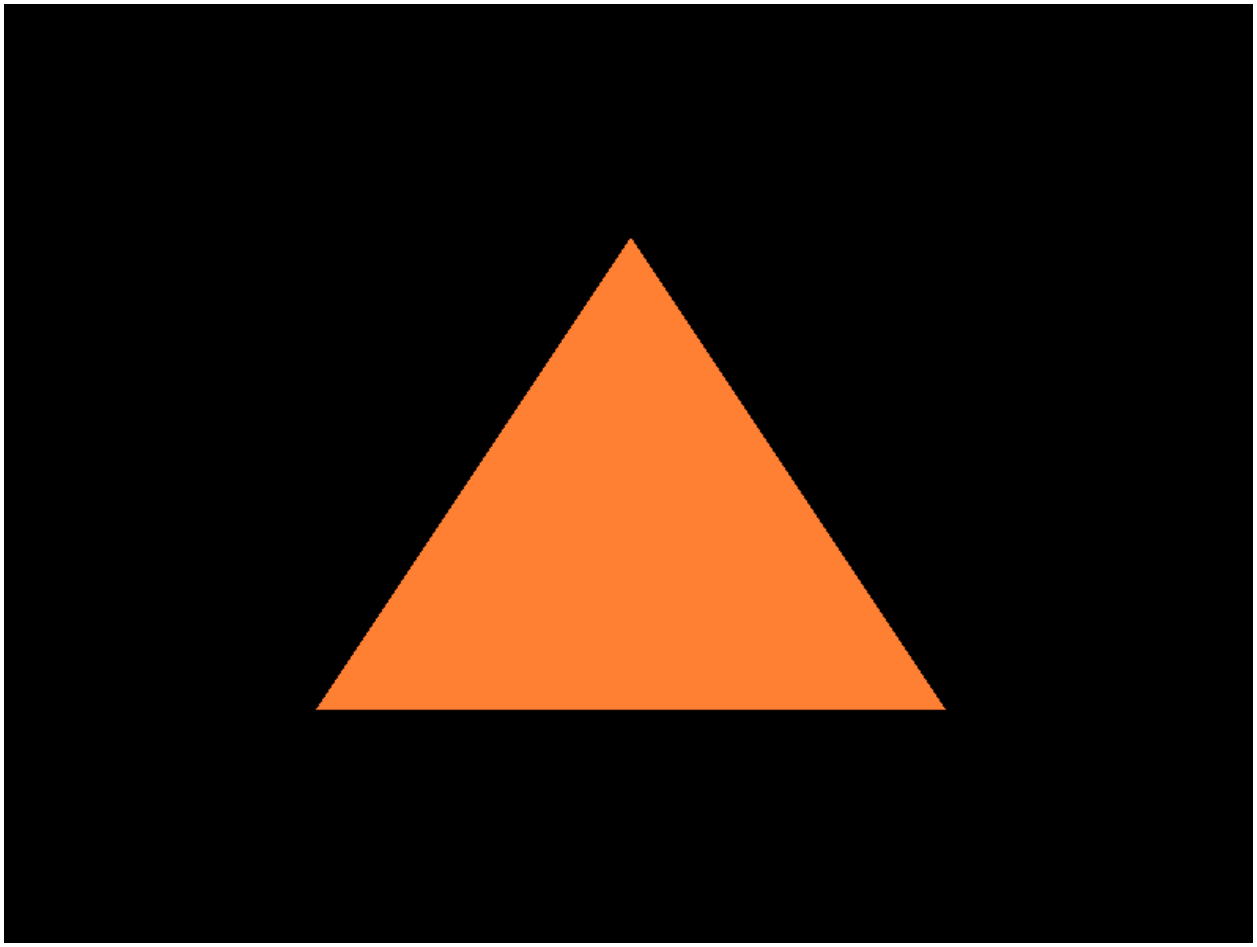


Рисунок 1 – Пример работы

4. Выводы

В процессе лабораторной работы был создан треугольник с помощью набора вершин и фрагментного шейдера, который придал оранжевый цвет.